

Fl.Shared.Utils.Components

Shared component library for .NET web applications. Provides reusable extensions, middleware and configurations for logging, exception handling, routing and service description.

Project structure

```
Fl.Shared.Utils.Components/  
├─ Constants.cs  
├─ Logging/  
│   └─ Models/  
│       └─ LoggingConfiguration.cs  
│   └─ Extensions/  
│       ├── Builder/  
│       ├── Either/  
│       ├── Error/  
│       ├── HostBuilder/  
│       └─ LoggerConfiguration/  
└─ Web/  
    ├── Endpoints/  
    ├── Extensions/  
    │   ├── ServiceCollection/  
    │   └─ WebApplication/  
    └─ Middlewares/  
        ├── ExceptionHandler/  
        ├── RequestLogContext/  
        ├── ServiceDescription/  
        └─ TraceLogContext/
```

Logging

Models

SerilogConfiguration

Configuration record for Serilog, bindable from `appsettings.json`.

Property	Type	Default	Description
Environment	string	""	Environment name (e.g. <code>production</code>)
System	string	""	Application system name

Property	Type	Default	Description
Customer	string	""	Customer identifier
MinimumLevel	LogEventLevel	Information	Minimum log level
Overrides	IDictionary<string, LogEventLevel>	{}	Per-namespace log level overrides

Extensions

LoggerConfigurationExtensions

Extensions on `Serilog.LoggerConfiguration` to apply default settings.

```
// Configure with Information level and CompactJson console output
loggerConfig.ConfigureDefault();

// Configure with a specific level
loggerConfig.ConfigureDefault(LogEventLevel.Debug);

// Configure with a full SerilogConfiguration (also enriches with Environment,
// System, Customer)
loggerConfig.ConfigureDefault(serilogConfig);
```

HostBuilderExtensions

Extensions on `IHostBuilder` to add Serilog.

```
// From a SerilogConfiguration instance
hostBuilder.AddSerilogLogging(serilogConfig);

// From IConfiguration (binds SerilogConfiguration automatically)
hostBuilder.AddSerilogLogging(configuration);
```

LoggingWebApplicationBuilderExtensions

Extension on `WebApplicationBuilder` to add Serilog by reading configuration automatically.

```
builder.AddSerilogLogging();
```

EitherLoggingExtensions

Extensions on `Either<Error, T>` and `EitherAsync<Error, T>` to log the left (error) branch without breaking the functional chain.

```
// With an explicit template
either.TeeLog(logger, "{Component} returned {Message}", componentName, message);

// With only the component name (uses default template)
either.TeeLog(logger, "MyComponent");

// Also available for EitherAsync
eitherAsync.TeeLog(logger, "MyComponent");
```

ErrorExtensions

Extensions on `LanguageExt.Common.Error` to convert an error into a logging `Action`.

```
// With a custom template
var logAction = error.ToLoggingAction(logger, "{Component} error: {Message}",
componentName, error.Message);

// With component name only (uses default template "{Component} raised an error
with {Message}")
var logAction = error.ToLoggingAction(logger, "MyComponent");
```

Web Endpoints

IEndpoint

Interface for defining endpoints via minimal API.

```
public interface IEndpoint
{
    WebApplication DefineEndpoints(WebApplication app);
}
```

Implement this interface and register implementations via `AddEndpointDefinitions`.

Middleware

JsonExceptionHandlerMiddleware

Catches any unhandled exception in the pipeline and returns an HTTP 500 JSON response containing the error message.

```
{  
  "message": "Error description"  
}
```

Registration:

```
services.AddExceptionHandler();  
// ...  
app.UseJsonExceptionHandler();
```

RequestLogContextMiddleware

Enriches the Serilog log context with a `CorrelationId` property for every HTTP request, reading it from `ICorrelationContextAccessor`.

Registration:

```
services.AddRequestLogContext();  
// ...  
app.UseRequestLogContext();
```

TraceLogContextMiddleware

Enriches the Serilog log context with `SpanId` and `TraceId` properties from the current `Activity` (W3C trace context).

Registration:

```
services.AddTraceLogContext();  
// ...  
app.UseTraceLogContext();
```

ServiceDescriptionMiddleware

Exposes service metadata (name and version) at the `GET /internal/description` endpoint. All other requests are forwarded to the next middleware.

Example response:

```
{
  "name": "my-service",
  "version": "1.0.0"
}
```

Registration:

```
services.AddServiceDescription(configuration);
// ...
app.UseServiceDescription();
```

Configuration is loaded from `servicedescription.json` (or a custom file):

```
builder.AddServiceDescriptionConfiguration(); //
servicedescription.json (default)
builder.AddServiceDescriptionConfiguration("custom-file.json"); // custom file
```

IServiceCollection extensions

ServiceCollectionExtensions

```
// Registers all default components in a single call:
// - JsonExceptionHandlerMiddleware
// - RequestLogContextMiddleware
// - ServiceDescriptionMiddleware
// - CorrelationId
// - RoutingConfiguration
// - JsonStringEnumConverter
services.AddDefaultComponents(configuration);

// Registers a strongly-typed configuration object read from IConfiguration
services.AddFromAppConfiguration<MyConfig>(configuration);
```

RoutingBasePathServiceCollectionExtensions

Registers `RoutingConfiguration` to configure an optional base path.

```
services.AddRoutingBasePath(configuration);
```

JsonSerializationServiceCollectionExtensions

Adds `JsonStringEnumConverter` to the JSON serialization options.

```
services.AddJsonSerializationConfiguration();
```

EndpointServiceCollectionExtensions

Scans the specified assemblies and registers all `IEndpoint` implementations as singletons.

```
// From marker types
services.AddEndpointDefinitions(typeof(MyMarker));

// From assemblies
services.AddEndpointDefinitions(Assembly.GetExecutingAssembly());
```

WebApplication extensions

ApplicationBuilderExtensions

Registers the default middleware pipeline in the correct order.

```
app.UseDefaultMiddlewares();
// Equivalent to:
// app.UseServiceDescription()
// app.UseCorrelationId()
// app.UseRequestLogContext()
// app.UseJsonExceptionHandler()
```

RoutingBasePathWebApplicationExtensions

Configures the routing base path if `RoutingConfiguration.PathBase` is set.

```
app.UseRoutingBasePath();
```

EndpointWebApplicationExtensions

Calls `DefineEndpoints` on all `IEndpoint` implementations registered in the container.

```
app.UseEndpointDefinitions();
```

Tests

Unit tests are located in `tests/Fl.Shared.Utils.Components.Unit.Tests` and cover:

Area	Class under test	Scenarios
Web / Middleware	<code>JsonExceptionHandlerMiddleware</code>	Exception in delegate → 500 + JSON error message; no exception → next delegate invoked
Web / Middleware	<code>ServiceDescriptionMiddleware</code>	Request to <code>/internal/description</code> → 200 + JSON with name and version; other path → next delegate invoked
Logging / Extensions	<code>EitherLoggingExtensions</code>	Error logged on the left branch of <code>Either</code> and <code>EitherAsync</code>
Logging / Extensions	<code>ErrorExtensions</code>	<code>Error</code> converted to a logging <code>Action</code> , with and without an associated exception

Main dependencies

- [Serilog](#) — structured logging
- [LanguageExt](#) — functional types (`Either`, `Error`, etc.)
- [CorrelationId](#) — correlation ID propagation
- [Scrutor](#) — assembly scanning for `IEndpoint`
- `Fl.Functional.Utils` — functional utilities (Tee, Map, MakeOption, etc.)
- `Fl.Configuration.Extensions` — extensions for reading strongly-typed configurations

Fl.Shared.Utils.Components.Web.Models

Shared model library for .NET web applications. Contains the data transfer objects and configuration records used across `Fl.Shared.Utils.Components` and `Fl.Shared.Utils.Mappers`.

Project structure

```
Fl.Shared.Utils.Components.Web.Models/  
├─ ErrorResponse.cs  
├─ RoutingConfiguration.cs  
└─ ServiceDescription.cs
```

Models

ErrorResponse

Immutable record representing a JSON error response body.

```
public record ErrorResponse(string Message);
```

Property	Type	Description
Message	string	Human-readable error description

Serialises to:

```
{  
  "message": "Something went wrong"  
}
```

Used by `JsonExceptionHandlerMiddleware` and `DataHttpContentMapper<T>` to produce consistent error payloads.

RoutingConfiguration

Configuration record for setting an optional URL base path for the application.

```
public record RoutingConfiguration  
{
```

```

    public static readonly RoutingConfiguration Default = new() { PathBase =
string.Empty };

    public string PathBase { get; init; } = string.Empty;
}

```

Property	Type	Default	Description
PathBase	string	""	Base path prefix for all routes (e.g. /api/v1)

RoutingConfiguration.Default is used as a fallback when no configuration entry is present. Consumed by RoutingBasePathServiceCollectionExtensions and RoutingBasePathWebApplicationExtensions in Fl.Shared.Utills.Components.

Example appsettings.json entry:

```

{
  "RoutingConfiguration": {
    "PathBase": "/api/v1"
  }
}

```

ServiceDescription

Configuration record that describes a service, exposed at the GET /internal/description endpoint.

```

public record ServiceDescription
{
    public string Name    { get; init; } = string.Empty;
    public string Version { get; init; } = string.Empty;
}

```

Property	Type	Default	Description
Name	string	""	Service name
Version	string	""	Service version

Serialises to:

```
{  
  "name": "my-service",  
  "version": "1.0.0"  
}
```

Populated from `servicedescription.json` via `AddServiceDescriptionConfiguration` and consumed by `ServiceDescriptionMiddleware`.

Example `servicedescription.json`:

```
{  
  "ServiceDescription": {  
    "Name": "my-service",  
    "Version": "1.0.0"  
  }  
}
```

Dependencies

This project has no external package dependencies. It targets `net8.0`, `net9.0` and `net10.0`.

Fl.Shared.Utls.Mappers

Shared mapper library for .NET web applications. Provides generic mapping abstractions and a ready-made implementation that converts functional types (`Option`, `Error`) into ASP.NET Core `ActionResult` HTTP responses.

Project structure

```
Fl.Shared.Utls.Mappers/  
├─ IMapper.cs  
├─ IApiResponseMapper.cs  
└─ DataHttpContentMapper.cs
```

Interfaces

`IMapper<T1, T2>`

Generic mapping contract between two types.

```
public interface IMapper<in T1, out T2>  
{  
    T2 Map(T1 item);  
}
```

Use this interface to define any single-direction mapping between an input type `T1` and an output type `T2`.

`IApiResultMapper<T>`

Specialisation of `IMapper` for mapping API results to ASP.NET Core `ActionResult`. Combines two mapping contracts:

- `IMapper<Option<T>, IActionResult>` — maps an optional value to an HTTP response
- `IMapper<Error, IActionResult>` — maps a functional error to an HTTP response

```
public interface IApiResponseMapper<T> :  
    IMapper<Option<T>, IActionResult>,  
    IMapper<Error, IActionResult>  
{ }
```

Implementations

DataHttpContentMapper<T>

Default implementation of `IApiResultMapper<T>`. Maps `Option<T>` and `Error` values to JSON `IResult` responses with appropriate HTTP status codes.

Input	HTTP status	Response body
<code>Option<T>.Some(value)</code>	200 OK	JSON-serialised <code>value</code>
<code>Option<T>.None</code>	404 Not Found	<code>{ "message": "Not Found" }</code>
<code>Error</code>	500 Internal Server Error	<code>{ "message": "Internal Server Error" }</code>

Error responses use `ErrorResponse` (from `Fl.Shared.Utills.Components.Web.Models`) with the reason phrase derived from the HTTP status code.

Usage

```
// Register in DI
services.AddSingleton<IApiResultMapper<MyDto>, DataHttpContentMapper<MyDto>>();

// Use in a minimal API endpoint
app.MapGet("/items/{id}", async (int id, IApiResultMapper<MyDto> mapper, IMyService
service) =>
{
    Option<MyDto> result = await service.GetById(id);
    return mapper.Map(result);
});
```

Tests

Unit tests are located in `tests/Fl.Shared.Utills.Mappers.Tests` and cover:

Class under test	Scenarios
<code>DataHttpContentMapper<T></code>	Error input → 500 JSON with "Internal Server Error"
<code>DataHttpContentMapper<T></code>	<code>Option.None</code> input → 404 JSON with "Not Found"
<code>DataHttpContentMapper<T></code>	<code>Option.Some(value)</code> input → 200 JSON with the original value

Main dependencies

- [LanguageExt](#) — functional types (`Option`, `Error`)
- `Microsoft.AspNetCore.App` — `IActionResult`, `Results`, `StatusCodes`
- `Fl.Functional.Utills` — functional utilities (`Map`, etc.)
- `Fl.Shared.Utills.Components.Web.Models` — `ErrorResponse`